SPRINT PERSONNALISÉ · AVANT-GOÛT STRATÉGIQUE

Semaine spéciale : tout JavaScript en 6 jours

Six jours chargés (25 août → 1er septembre) pour affronter les gros concepts durs pendant que tu as le temps. On accélère sur les bases, on prend le temps sur le cœur du langage, et on finit par un vrai premier contact React. Ce n'est pas la formation complète — c'est le défrichage intelligent avant de tout reprendre proprement le 1er septembre.

6 jours

5-6 h/jour (4 h minimum). On frappe fort maintenant, on affine après.

Le cœur dur

Fonctions, tableaux d'objets, map/filter/reduce, le DOM, l'async : les vrais verrous.

La carotte

Jour 6 : ton premier React. Le moment « ahh, c'est que ça ».

Zéro copier-coller

Tu tapes tout. La mémoire des doigts fait 50 % du boulot chez un débutant.

Profil : HTML/CSS fragile à moyen (assumé) · Windows (Git Bash) · gros créneaux cette semaine, puis 1h30-2h en semaine dès le 1er sept · ressources 100 % en français.

> **Le deal, clairement.** Cette semaine n'est pas là pour que tu « maîtrises » JavaScript — personne n'y arrive en 6 jours. Elle est là pour que tu voies de tes yeux *tout le paysage* : affronter les morceaux durs maintenant, avec du

> temps et mon aide en direct, plutôt que dans les créneaux serrés d'après. Le 1er septembre, tu reprends tout — mais avec des repères déjà posés, et donc beaucoup plus vite. C'est ça, la stratégie.

Mode d'emploi de la semaine

Même principe que ta roadmap : chaque soir tu m'écris « **Jour X spécial terminé** », je te pose 2-3 questions de vérification (concepts + un mini-défi de code, sans tes notes), et on ne passe au jour suivant que si c'est solide. La différence avec la roadmap normale : ici on assume d'aller vite sur les bases, parce que tu les **reverras en détail après le 1er septembre**. L'objectif n'est pas la perfection, c'est la **familiarité**.

Ta journée type (adaptée à tes créneaux)

Bloc	Durée (jour 5-6 h)	Contenu
1. Découverte active	1 h 30 – 2 h	Cours, docs, vidéos du jour — éditeur ouvert en permanence, tu tapes chaque exemple. Interdit de regarder une vidéo sans coder en même temps.
2. Pratique	2 h 30 – 3 h	Les exercices du jour, tutoriel fermé. C'est ici que ça rentre vraiment. Bloqué > 25 min sur un point : tu notes la question, tu contournes, tu avances — et tu me la poses.
3. Consolidation	30 min	Refais un truc du matin de mémoire, relis tes notes, prépare tes questions, commit Git du jour.

> **Jour à 4 h seulement ?** Pas de panique. Tu gardes le bloc 2 (pratique) en priorité absolue et tu réduis la découverte. Mieux vaut 4 h les mains dans le code que 6 h de vidéos passives. C'est écrit noir sur blanc dans ta roadmap, et c'est encore plus vrai cette semaine.

Les 4 règles de la semaine

- **Tape tout, ne copie rien.** Même si c'est plus lent. Surtout si c'est plus lent.
- **Familiarité > maîtrise.** Le but est de reconnaître et comprendre, pas de tout retenir. Le par-cœur viendra avec la répétition d'après.
- **Un dépôt GitHub pour la semaine.** Un dossier `semaine-speciale`, un commit par jour minimum. On garde la trace.
- **Chaque objectif se teste à voix haute.** Si tu ne peux pas l'expliquer simplement, tu ne l'as pas compris — et je le verrai aux questions du soir.

Vue d'ensemble des 6 jours

Jour 1	**JS de base en accéléré : variables, conditions, boucles** La console comme terrain de jeu · FizzBuzz · mini-programmes logiques
Jour 2	**Les fonctions : ranger et réutiliser sa logique** Ta première « boîte à outils » de fonctions · refacto des exos du jour 1
Jour 3	**Tableaux & objets : la structure reine du web** Tableaux d'objets (produits, clients) · CRUD en mémoire

Jour 4	**map / filter / reduce : LE verrou qui débloque React** Transformer, filtrer, agréger des listes sans boucle manuelle
Jour 5	**Le DOM + un avant-goût d'async : JS anime une vraie page** Générer du HTML depuis JS · écouter des clics · toucher à fetch
Jour 6	**Premier contact React : « c'est que du JS bien rangé »** Ton premier projet Vite + composants + props + un peu de state

JOUR 1 · 25 AOÛT

JavaScript de base, en accéléré

On démarre par le socle : parler à la machine, stocker des valeurs, prendre des décisions, répéter des actions. Tout se passe dans la **console** — pas de page web, on isole la logique pure. On va vite : ces bases, tu les reverras en profondeur après le 1er septembre. Aujourd'hui, on veut juste que ça te devienne familier.

1.1 Variables, types, et la console

≈ 2 h

CONCEPTS

La balise `<script>`, la console du navigateur (F12 → onglet Console), `console.log()`. Variables `let` et `const` (et pourquoi on évite `var`). Les types primitifs : string, number, boolean, `null`, `undefined`. Les template literals avec les backticks : `` `Bonjour ${nom}` ``.

RESSOURCES

- Grafikart, formation « Apprendre JavaScript » (chapitres variables et types) — grafikart.fr/formations/formation-javascript
- MDN, « Premiers pas en JavaScript » (en français) — developer.mozilla.org

EXERCICE

Dans la console : déclare une variable de chaque type, affiche des phrases dynamiques avec les template literals (ex. `` `Je m'appelle ${prenom} et j'ai ${age} ans` ``). Provoque volontairement une erreur en réassignant une `const`, lis le message d'erreur en entier — apprendre à lire les erreurs commence maintenant.

> **Objectif mesurable**
> J'explique la différence entre `let` et `const`, je nomme les 5 types primitifs, et j'affiche une phrase construite avec des variables.

1.2 Opérateurs & conditions

≈ 1 h 30

CONCEPTS

Opérateurs arithmétiques et de comparaison. **La différence cruciale** `==` **vs** `===`. Opérateurs logiques `&&`, `||`, `!`. Les conditions `if` / `else if` / `else`, l'opérateur ternaire, le `switch`. Les valeurs « truthy » et « falsy ».

RESSOURCES

- Grafikart JS (chapitre conditions)
- fr.javascript.info, chapitres opérateurs et comparaisons — fr.javascript.info/operators (la traduction française du meilleur manuel JS du web)
- MDN, « Prendre des décisions dans le code »

EXERCICE

Mini-programmes console : pair ou impair · majeur/mineur à partir d'une année de naissance · un exemple piège de `==` vs `===` que tu expliques. Puis passe à l'exercice phare ci-dessous.

> **Objectif mesurable**
> J'explique == vs === avec un exemple qui piège, et je choisis la bonne condition selon la situation.

1.3 Boucles + le classique FizzBuzz

≈ 1 h 30

CONCEPTS

`for`, `while`, `break` / `continue`, compteurs et accumulateurs. Comment choisir sa boucle.

RESSOURCES

- Grafikart JS (chapitre boucles)
- MDN, « Les boucles dans le code » (FR)

EXERCICE

Table de multiplication d'un nombre · somme des nombres de 1 à 100 · et **FizzBuzz** : affiche les nombres de 1 à 100, mais « Fizz » pour les multiples de 3, « Buzz » pour ceux de 5, « FizzBuzz » pour les deux. C'est LE test d'entrée universel du métier — si tu le sors sans aide, tes bases logiques tiennent.

> **Objectif mesurable**
> J'écris FizzBuzz seul en moins de 10 minutes et j'explique pourquoi j'ai choisi telle boucle.

AUTO-CONTRÔLE DU SOIR — JOUR 1

- ✓ Différence entre `let` et `const` ? Entre == et === ?
- ✓ Que vaut `"5" == 5` ? Et `"5" === 5` ? Pourquoi ?
- ✓ Défi : réécrire FizzBuzz de mémoire (10 min).

JOUR 2 · 26 AOÛT

Les fonctions : ranger et réutiliser

Une fonction, c'est une boîte : tu lui donnes des ingrédients (paramètres), elle te rend un résultat (`return`). C'est le concept qui transforme un « script bordélique » en code propre et réutilisable — et c'est la brique de base absolue de React. Journée dense mais très gratifiante : à la fin, tu ranges ta logique comme un pro.

2.1 Déclarer et appeler des fonctions

≈ 2 h

CONCEPTS

Déclaration vs expression de fonction. Paramètres et arguments. **`return` (et sa différence capitale avec `console.log` !)**. La portée des variables (scope) : pourquoi une variable née dans une fonction n'existe pas dehors. Les paramètres par défaut.

RESSOURCES

- Grafikart JS (chapitre fonctions)
- MDN, « Fonctions — réutiliser du code » (FR)
- fr.javascript.info, chapitre « Fonctions » — fr.javascript.info/function-basics

EXERCICE

Reprends tes exercices du Jour 1 et transforme-les en fonctions : `estPair(nombre)`, `estMajeur(age)`, `fizzbuzz()`. Teste la différence : une fonction avec `return` vs une avec `console.log` — essaie de réutiliser le résultat de chacune, tu vas comprendre pourquoi `return` est roi.

> Objectif mesurable
> J'explique la différence entre `return` et `console.log`, et pourquoi une variable de fonction n'existe pas à l'extérieur.

2.2 Fonctions fléchées + ta boîte à outils

≈ 2 h 30

CONCEPTS

Les fonctions fléchées `=>` (la syntaxe que React utilise en permanence). Comparaison avec la syntaxe classique. Une fonction qui en appelle une autre. Écrire du code découpé en petites fonctions courtes.

RESSOURCES

- fr.javascript.info, « Fonctions fléchées, les bases » — fr.javascript.info/arrow-functions-basics
- Grafikart JS (chapitre fonctions, partie ES6)

EXERCICE

Crée ta **bibliothèque de 5 fonctions utilitaires**, chacune en version fléchée : `calculTTC(prixHT)` (ajoute 20 %), `formatPrix(nombre)` (renvoie `"12,50 €"`), `estMajeur(age)`, `genererPseudo(prenom, annee)`, `max(a, b)` (sans `Math.max`). Écris-les d'abord en fonction classique, puis réécris-les en fléchées : voir les deux côte à côte, c'est ça qui fait cliquer la syntaxe.

> Objectif mesurable
> J'écris une fonction fléchée avec valeur de retour sans hésiter, et je lis sans effort `const double = n => n * 2`.

Pourquoi cette journée compte double. 80 % de React, c'est écrire des petites fonctions fléchées qui reçoivent des données et renvoient quelque chose. Si les fonctions te deviennent naturelles aujourd'hui, le Jour 6 te paraîtra presque facile. C'est un investissement direct.

AUTO-CONTRÔLE DU SOIR — JOUR 2

- ✓ Quelle est la différence entre `return` et `console.log` ?
- ✓ Pourquoi une variable déclarée dans une fonction n'est-elle pas accessible dehors ?
- ✓ Défi : écrire `formatPrix(nombre)` en fonction fléchée, de mémoire (10 min).

JOUR 3 · 27 AOÛT

Tableaux & objets : la structure reine

On arrive au cœur du sujet. Presque toute donnée sur le web est un **tableau d'objets** : une liste de produits, de clients, de réservations. Maîtriser cette structure, c'est comprendre 90 % de ce que manipule une application. On prend le temps sur cette journée — c'est un des vrais verrous.

3.1 Les tableaux

≈ 2 h

CONCEPTS

Création, l'index qui commence **à 0**, `.length`. Ajouter/retirer : `push`, `pop`, `shift`, `unshift`. Chercher : `indexOf`, `includes`. Parcourir avec `for...of` et `forEach`. Survol de `slice` vs `splice`.

RESSOURCES

- Grafikart JS (chapitre tableaux)
- MDN, « Les tableaux » (FR)
- fr.javascript.info, « Les tableaux » — fr.javascript.info/array

EXERCICE

Gestionnaire de courses en console : un tableau, puis des fonctions `ajouter(article)`, `retirer(article)`, `afficher()`. Trouve le min et le max d'un tableau de prix sans `Math.min`. Parcours un tableau avec `for...of` ET avec `forEach` pour sentir la différence.

> Objectif mesurable
> J'ajoute, je retire et j'affiche les éléments d'un tableau sans regarder la doc pour les méthodes de base.

3.2 Les objets

≈ 1 h 30

CONCEPTS

Propriétés et valeurs. Notation point (`client.nom`) vs crochets (`client["nom"]`). Objets imbriqués (`client.adresse.ville`). Ajouter/modifier une propriété. Introduction au format JSON (tu verras que c'est juste des objets en texte).

RESSOURCES

- Grafikart JS (chapitre objets)
- MDN, « Les bases des objets JavaScript » (FR)

EXERCICE

Crée un objet `client` avec nom, email, budget, et une adresse imbriquée. Accède à chaque propriété, même la ville imbriquée. Modifie le budget. Ajoute une propriété `telephone` après coup.

> Objectif mesurable
> J'accède à n'importe quelle propriété d'un objet, même profondément imbriquée, et je sais la modifier.

3.3 Tableaux d'objets — le combo qui change tout

≈ 2 h

CONCEPTS

La structure reine : un tableau qui contient des objets. Le parcourir, accéder à la propriété de chaque élément, filtrer « à la main » avec une boucle et une condition (avant d'automatiser ça au Jour 4).

EXERCICE

Modélise 5 clients : `[{nom, email, budget}, …]`. Écris une fonction qui parcourt le tableau et affiche une fiche formatée par client. Écris une deuxième fonction qui n'affiche que les clients dont le budget dépasse un seuil (avec une boucle `for...of` + un `if` — garde ce code, tu le compareras à `filter` demain).

> Objectif mesurable
> Je crée et parcours un tableau d'objets, et je fais un CRUD basique (créer, lire, modifier, supprimer) en mémoire.

AUTO-CONTRÔLE DU SOIR — JOUR 3

- ✓ Pourquoi l'index d'un tableau commence-t-il à 0 ? Que renvoie `.length` ?
- ✓ Différence entre notation point et notation crochets pour un objet ?
- ✓ Défi : à partir d'un tableau d'objets `{nom, prix}`, afficher le nom du produit le plus cher (15 min).

JOUR 4 · 28 AOÛT

map / filter / reduce : le verrou React

La journée la plus importante de la semaine. Ces trois méthodes remplacent des boucles entières par une ligne claire — et surtout, *tout affichage de liste en React passe par `map`*. Si tu ressors ce soir en étant à l'aise avec map/filter/reduce, le passage à React sera une formalité. On y va en douceur, avec beaucoup de pratique.

4.1 map & filter

≈ 2 h 30

CONCEPTS

`map` : transformer chaque élément → **un nouveau tableau de même taille**. `filter` : ne garder que les éléments qui passent un test → **un tableau plus petit**. `find` : trouver le premier élément qui correspond. L'idée clé : ces méthodes prennent une **fonction** en argument (d'où l'importance du Jour 2).

RESSOURCES

- fr.javascript.info, « Méthodes de tableau » — fr.javascript.info/array-methods
- MDN, fiches `map` et `filter` (FR)
- Grafikart : cherche « tableaux » sur grafikart.fr pour les vidéos dédiées aux méthodes

EXERCICE

Reprends ton tableau de clients (Jour 3). Avec `map` : obtiens la liste des noms seuls. Avec `filter` : obtiens les clients au budget > 1000. **Puis compare ton `filter` avec ta boucle `for...of` d'hier** : même résultat, une ligne au lieu de six. C'est le déclic.

> Objectif mesurable
> J'utilise `map` et `filter` sans doc, et je sais dire ce que chacune renvoie (un nouveau tableau, filtré ou transformé).

4.2 reduce, sort & le chaînage

≈ 2 h 30

CONCEPTS

`reduce` : réduire un tableau à **une seule valeur** (un total, une moyenne) — le plus abstrait des trois, on le prend tranquillement. `sort` pour trier. Le **chaînage** : `filter(...).map(...)` à la suite, le combo que tu utiliseras toute ta carrière.

RESSOURCES

- fr.javascript.info, « Méthodes de tableau » (section reduce)
- MDN, fiche `reduce` (FR)

EXERCICE

Sur un tableau de 8 produits `{nom, prix, stock}` : le prix total du panier avec `reduce` · les produits triés par prix croissant avec `sort` · les noms des produits en stock uniquement, en chaînant `filter` puis `map`. Un résultat par ligne, propre.

> Objectif mesurable
> J'explique ce que fait `reduce` avec mes mots, et je chaîne `filter` + `map` pour obtenir une liste transformée.

Si reduce te résiste, c'est normal. C'est le concept le plus dur de la semaine pour tout le monde. L'objectif aujourd'hui n'est pas de le maîtriser, mais de *comprendre l'idée* : « je pars d'un tableau, j'accumule, je finis avec une seule valeur ». On le reverra. map et filter, en revanche, doivent être solides — ce sont eux qui servent tout le temps.

AUTO-CONTRÔLE DU SOIR — JOUR 4

- ✓ Que renvoie `map` ? Et `filter` ? Et `reduce` ?
- ✓ Donne un exemple réel d'usage de chacune.
- ✓ Défi : depuis un tableau de produits, afficher les noms de ceux en stock triés par prix (20 min).

JOUR 5 · 29 AOÛT (WEEK-END — VISE 5-6 H)

Le DOM + un avant-goût d'async

Ton JS sort enfin de la console et anime une **vraie page**. Le DOM, c'est le pont entre ta logique et ce que l'utilisateur voit. En bonus l'après-midi, on goûte à l'asynchrone et à `fetch` — juste assez pour comprendre comment une page va chercher des données ailleurs. On ne creuse pas l'async à fond (ce sera pour après le 1er sept), on le rend familier.

5.1 Sélectionner et modifier la page

≈ 2 h

CONCEPTS

Le DOM = l'arbre de la page vu par JS. `querySelector` / `querySelectorAll`. Modifier le texte avec `textContent` (et le risque de sécurité d'`innerHTML`). Jouer avec les classes via `classList` (`add`, `remove`, `toggle`). Créer des éléments : `createElement`, `append`.

RESSOURCES

- Grafikart JS (chapitres sur le DOM)
- MDN, « Manipuler des documents » (FR)

EXERCICE

Crée une petite page HTML. Depuis ton tableau d'objets clients (Jour 3), **génère une carte HTML par client entièrement en JavaScript** (createElement + append, ou un `map` qui construit du HTML). C'est le mécanisme au cœur de toutes les applis web — et exactement ce que React automatise.

> Objectif mesurable
> Je génère du HTML dynamiquement à partir d'un tableau d'objets, et j'affiche ces cartes sur une vraie page.

5.2 Les événements : réagir aux clics

≈ 1 h 30

CONCEPTS

`addEventListener`, l'objet `event`, les événements `click` et `input`. `preventDefault` (indispensable sur les formulaires). L'idée : « quand l'utilisateur fait X, exécute la fonction Y ».

RESSOURCES

- Grafikart JS (chapitre événements)
- MDN, « Introduction aux événements » (FR)

EXERCICE

Un compteur avec boutons +, −, reset qui affiche la valeur à l'écran. Un champ texte qui affiche ce que tu tapes en direct en dessous. Un bouton « ajouter un client » qui crée une carte de plus (relie ça à l'exercice 5.1).

> Objectif mesurable
> Je fais réagir une page à un clic et à une saisie clavier, avec ma propre fonction.

5.3 Avant-goût : async & fetch

≈ 1 h 30

CONCEPTS

Synchrone vs asynchrone (pourquoi le navigateur ne peut pas « attendre »). L'idée d'une **promesse** : un résultat qui arrivera plus tard. `fetch` pour aller chercher des données sur une API, `async` / `await`, et `try` / `catch` pour gérer les erreurs. On reste sur la compréhension, pas la maîtrise.

RESSOURCES

- MDN, « Utiliser l'API Fetch » (FR)
- Grafikart : cherche « fetch » et « promesses » sur grafikart.fr
- API gratuite sans clé pour t'entraîner : TheMealDB — themealdb.com/api.php

EXERCICE

Un bouton « Charger une recette » qui va chercher une recette aléatoire sur TheMealDB avec `fetch` + `async/await`, et affiche son nom et son image sur la page. Affiche « Chargement… » pendant la requête. Coupe ton wifi et observe l'erreur gérée par `try/catch`. Si ça marche : tu viens de faire ta première app qui parle à internet.

> **Objectif mesurable**
> Je récupère des données d'une API publique et je les affiche sur ma page, avec un état de chargement. Je sais expliquer ce qu'est une promesse avec mes mots.

AUTO-CONTRÔLE DU SOIR — JOUR 5

- ✓ Différence entre `textContent` et `innerHTML`, et quel risque pose le second ?
- ✓ À quoi sert `addEventListener` ? Et `preventDefault` ?
- ✓ Explique `async/await` à quelqu'un qui ne code pas, en 3 phrases.

JOUR 6 · 30-31 AOÛT (WEEK-END) — LA RÉCOMPENSE

Premier contact React

Le grand moment. Tout ce que tu as vu cette semaine (fonctions fléchées, tableaux d'objets, `map`) est **exactement** ce que React manipule. L'objectif n'est pas de « savoir React » ce soir, mais de vivre le déclic : « ah, c'est juste du JavaScript bien rangé ». La doc officielle fr.react.dev est en français et excellente — ce sera ta ressource après le 1er sept.

6.1 Créer un projet React + comprendre JSX

≈ 2 h 30

CONCEPTS

Pourquoi React (l'interface = une fonction des données). Créer un projet avec Vite : `npm create vite@latest` (choisis React + JavaScript). Lancer le serveur de dev. Le **JSX** : écrire du HTML dans du JavaScript, les 3 règles de base (une seule racine, `className` au lieu de `class`, expressions entre `{ }`). Un composant = une fonction qui renvoie du JSX.

RESSOURCES

- fr.react.dev, « Démarrage rapide » puis « Votre premier composant » — fr.react.dev/learn
- Doc Vite, guide de démarrage — vite.dev/guide
- Grafikart, « Apprendre React » en complément vidéo

EXERCICE

Crée ton projet Vite. Dans `App.jsx`, remplace tout par un titre et un paragraphe à toi. Puis crée un composant `Carte.jsx` (une fonction qui renvoie une petite carte en JSX) et affiche-le dans `App`. Fais afficher une expression JavaScript entre accolades (ex. `{2 + 2}` ou `{new Date().getFullYear()}`).

> **Objectif mesurable**
> Je crée un projet React depuis zéro, j'écris un composant, et j'explique les 3 règles de base du JSX.

6.2 Props + afficher une liste avec map

≈ 2 h 30

CONCEPTS

Les **props** : passer des données d'un composant parent à son enfant (comme des arguments de fonction !). Afficher une **liste avec** `map` et l'attribut `key`. Un tout premier aperçu de `useState` pour rendre un bouton interactif — juste pour voir la magie de la mise à jour automatique.

RESSOURCES

- fr.react.dev, « Passer des props à un composant » et « Afficher des listes » — fr.react.dev
- fr.react.dev, « L'état : la mémoire d'un composant » (survol de `useState`)

EXERCICE

Le projet qui fait tout cliquer : reprends ton tableau de clients (ou de produits). Crée un composant `Carte` qui reçoit ses infos en props. Dans `App`, fais `tableau.map(...)` pour afficher une `Carte` par élément. Compare mentalement avec ton exercice DOM du Jour 5 : **même résultat, dix fois moins de code**. Bonus : un bouton compteur avec `useState`.

> Objectif mesurable
> J'affiche une liste de cartes depuis un tableau d'objets avec `map` et des props. Et surtout : je *vois* que React n'est que du JS que je connais déjà, mieux organisé.

Le vrai objectif de ce jour. Pas de tout comprendre — d'avoir le déclic. Quand tu réaliseras que `tableau.map(client => <Carte ... />)`, c'est juste le `map` du Jour 4 qui renvoie du JSX au lieu de texte, tu auras gagné ta semaine. Le reste de React se construira là-dessus, sans stress, après le 1er septembre.

AUTO-CONTRÔLE DU SOIR — JOUR 6

- ✓ Qu'est-ce qu'un composant React ? Qu'est-ce qu'une prop ?
- ✓ Pourquoi affiche-t-on une liste avec `map` en React ? Où as-tu déjà utilisé `map` cette semaine ?
- ✓ Défi : ajouter une `Carte` supplémentaire au tableau et vérifier qu'elle s'affiche (15 min).

Après le sprint : la reprise du 1er septembre

Tu as désormais touché à tout le paysage : logique, fonctions, données, DOM, async, React. Rien n'est « maîtrisé » — et c'est parfaitement normal, ce n'était pas le but. Le but était de **défricher**, pour que la reprise sérieuse soit rapide et sans angoisse.

✓ Ce que tu dois pouvoir dire au soir du Jour 6

- « Je sais écrire une condition, une boucle, une fonction fléchée sans réfléchir trop longtemps. »
- « Un tableau d'objets ne me fait plus peur, et je sais le parcourir. »
- « Je comprends ce que font `map` et `filter`, et je les reconnais dans du code. »
- « J'ai fait afficher des données sur une vraie page, et même depuis une API. »
- « J'ai créé un projet React et affiché une liste — je vois que c'est du JS organisé. »

> **Si un de ces points cloche**, ce n'est pas grave du tout : on le note comme priorité n°1 de la reprise. On ne se raconte pas d'histoires, on répare — c'est exactement l'esprit de ton protocole de validation.

→ Comment on reprend, dès le 1er septembre

LE PLAN

On rouvre ta roadmap officielle à la **Semaine 1** et on la déroule proprement — mais avec un énorme avantage : tu ne découvres plus, tu *approfondis*. Ce qui prenait une journée entière pourra parfois se faire plus vite, ce qui libère du temps pour la pratique et les projets. Tes créneaux de 1h30-2h en semaine et 4h le week-end suffisent, parce que tu ne pars plus de zéro sur les concepts durs.

RYTHME DE REPRISE PROPOSÉ

- **En semaine (1h30-2h) :** 30-40 min de découverte/consolidation d'un concept, le reste en pratique pure (tu tapes, tu construis).
- **Le week-end (4h) :** la séance projet — c'est là que tu avances sur tes livrables de portfolio.
- **Le protocole reste le même :** « Jour X terminé » chaque soir, questions de validation, on n'avance que si c'est solide.

> L'état d'esprit
> Tu t'es remis en selle après avoir lâché les défis — ça, c'est la seule chose qui compte vraiment. La régularité bat l'intensité. 1h30 tous les jours pendant des mois construit un développeur ; 12h un dimanche puis rien ne construit rien. On tient le cap ensemble, un jour après l'autre.